FACULTAD DE INFORMÁTICA
MÁSTER DE INGENIERÍA DEL SOFTWARE



SolidarianID

Proyecto final

Grupo 3:
Antonio Pérez Serrano
Fabián Sola Durán
Manuel Bernabé Rodríguez
Pablo Peris Solano
Sergio Ros Liarte

Prácticas Continuas
12/01/2026

Índice

1. Introducción **2**

2. Usuario de AWS **2**

3. Creación de un *pipeline* de integración continua **3**
3.1. Fase preparatoria 4
3.2. Fase de formato 5
3.3. Fase de compilación 6
3.4. Fase de *testing* 6
3.5. Fase de *SonarQube* 7

4. Diseño de un pipeline de despliegue **9**
4.1. Stacks desplegados 10
4.1.1. lab-network.yaml 10
4.1.2. stack.yaml 12
4.2. Fase de preparación 15
4.3. Fase de *Docker* 16
4.3.1. Construcción de la imagen Docker 16
4.3.2. Envio de las imágenes a DockerHub 17
4.4. Fase de despliegue en AWS 18
4.4.1. Configuración de credenciales 21
4.4.2. Despliegue de infraestructura 22
4.4.3. Actualización de contenedores y validación 23

5. Conclusiones **25**

1. Introducción

Los *pipelines* de integración continua (*CI*) y entrega continua (*CD*) son componentes fundamentales en la arquitectura de un desarrollo moderno de *software*. Estos automatizan los procesos de validación, construcción y despliegue del código, garantizando que cada cambio sea probado exhaustivamente antes de llegar a producción y que las nuevas versiones se distribuyan de forma segura y eficiente.

El *pipeline CI*, ejecutado en la rama *develop*, verifica la calidad del código mediante *linting, testing* y análisis estático con *SonarQube*, asegurando que solo código de alta calidad sea integrado.

El *pipeline CD*, activado en la rama *main*, automatiza la construcción de *imágenes Docker*, su distribución en *Docker Hub* y el despliegue completo en infraestructura *AWS*, permitiendo que las nuevas versiones lleguen a producción de manera ágil y confiable.

Juntos, estos *pipelines* implementan prácticas de *DevOps* que aceleran el desarrollo, reducen errores humanos y proporcionan visibilidad sobre el estado de la aplicación en cada etapa del ciclo de vida.

En la bibliografía se detalla el enlace [1] a nuestro repositorio de GitHub en el que se puede encontrar la carpeta `.github/workflows` con los distintos detalles relativos a la asignatura.

2. Usuario de AWS

Para el acceso a la plataforma de *Amazon Web Services* (AWS) por parte del profesorado, se han habilitado las siguientes credenciales de usuario. Cabe destacar que no se ha utilizado el laboratorio de prácticas proporcionado, sino que el integrante *Antonio Pérez* ha creado una cuenta propia de AWS desde la cual se han realizado los despliegues del proyecto. Con el objetivo de seguir buenas prácticas de seguridad, no se ha empleado la cuenta de usuario *root*; en su lugar, se ha creado un usuario *IAM* con un rol de administrador, otorgándole acceso completo a los servicios y recursos necesarios para la ejecución del proyecto.

```
URL de acceso: https://866064992971.signin.aws.amazon.com/console
Usuario: ProfesorAWS
Contraseña: Umu@Aws_Profesor
Región: eu-south-2
```

Para la correcta verificación de los *stacks* de *CloudFormation*, se debe seleccionar la región `eu-south-2`. Para la corrección y/o auditoría del despliegue, se proporcionarán las claves `.pem` asociadas a **ambas instancias** (proxy y aplicaciones) con el objetivo de que el profesorado pueda acceder por SSH si fuese necesario (por ejemplo, para verificar logs, configuración o estado de contenedores). Las claves privadas utilizadas para el acceso

remoto por *SSH* a las instancias EC2 son `solidarianid.pem`, correspondiente a la instancia *proxy*, y `solidarian-keypair.pem`, asociada a la instancia EC2 donde se ejecutan las aplicaciones. El uso y la gestión de ambas claves, así como el procedimiento de acceso a las instancias, se explicarán en detalle en el apartado de despliegue. Una vez finalizado el proceso de evaluación y calificada la práctica, dicho rol será eliminado por motivos de seguridad.

3. Creación de un *pipeline* de integración continua

La creación de un *pipeline* de integración continua en nuestro proyecto ha resultado ser una de las prácticas más importantes para asegurar el progreso de nuestro software. Para ello, hemos utilizado la herramienta *Github Actions* con el fin de asegurar la calidad del código en cada *pull request* que se realizaba a nuestra rama *develop*. El fichero donde se encuentra dicho *CI* está en la siguiente ruta:

`SolidarianID/.github/workflows/ci.yaml`

El archivo `ci.yaml` define un *pipeline* automatizado para el proyecto *SolidarianID*, el cual se ejecuta automáticamente ante cada *push* o *pull request* dirigidos a la rama *develop*. Su objetivo principal es garantizar la calidad del código, la correcta compilación del proyecto y el cumplimiento de los estándares definidos, actuando como una barrera de validación previa al posterior despliegue en la rama principal (*main*).

Con respecto al uso de *actions* se tuvo en cuenta por motivos de seguridad la conveniencia de anclar las dependencias del *pipeline* a identificadores inmutables, utilizando el *hash* del *commit* en las *GitHub Actions* y el *digest* (`sha256`) en las imágenes Docker en lugar de referencias móviles como `@v4` o *tags* de versión. Este enfoque garantiza la inmutabilidad y reproducibilidad de las ejecuciones, reduce el riesgo de *supply-chain attacks* al evitar la incorporación automática de cambios no auditados, facilita la trazabilidad y auditoría de los artefactos utilizados y permite un control explícito de las actualizaciones, asegurando que el *pipeline* ejecute exactamente las versiones verificadas por el equipo.

Durante la fase de desarrollo, se han eliminado los privilegios de *push* directo sobre la rama *main* con el fin de asegurar que el código contenido en ella sea funcional y estable. De forma análoga, para reforzar la seguridad y estabilidad de la rama *develop*, tampoco se permite realizar *pushes* directos por parte de los desarrolladores del equipo, salvo modificaciones puntuales efectuadas por el propietario del repositorio.

En su lugar, los desarrolladores deben trabajar sobre ramas alternativas basadas en *develop* y solicitar una *pull request* para integrar sus cambios. Una vez superado satisfactoriamente el proceso de *CI*, se activa el flujo de revisión, en el cual se han definido políticas de *merge* que obligan a que, al menos, dos revisores aprueben la solicitud, el *pipeline* de *CI* finalice correctamente y no existan conflictos antes de permitir la integración definitiva del código.

El *pipeline* se divide en varias fases:

1. **Preparatoria**: en esta etapa se descarga el código del repositorio y se instala lo necesario para ejecutar el *pipeline*.

2. **Formato**: evaluación del formato con *Prettier* y *Linting* para seguir los estándares del equipo.

3. **Compilación**: compilación del proyecto para verificar que no hay errores de *TypeScript* o construcción.

4. ***Testing***: ejecución de toda la *suite* de pruebas (unitarias, integración, e2e, etc) con cobertura de código, garantizando que los nuevos cambios no rompan funcionalidades existentes.

5. ***SonarQube***: realiza un análisis exhaustivo de calidad de código con *SonarQube*, detectando *code smells*, vulnerabilidades de seguridad, duplicaciones y otros problemas técnicos.

Estas fases se explican a continuación en los siguientes apartados.

3.1. Fase preparatoria

La etapa preparatoria descarga el código del repositorio, instala *Node.js 24*, cachea las dependencias *npm* para acelerar futuras ejecuciones, e instala todas las herramientas necesarias (*Prettier*, *ESLint*, *Jest*, etc.) que el *pipeline* requiere para ejecutarse. Sin esto, no habría código ni herramientas para analizar. Estas 3 etapas podemos observarlas en el trozo de código 1.

```yaml
- name: Checkout code
  uses: actions/checkout@8e8c483db84b4bee98b60c0593521ed34d9990e8
  ↪  #actions/checkout@v6.0.1
  with:
    fetch-depth: 0

- name: Setup Node.js
  uses: actions/setup-node@395ad3262231945c25e8478fd5baf05154b1d79f
  ↪  #actions/setup-node@v6.0.1
  with:
    node-version: ${{ matrix['node-version'] }}
    cache: npm
    cache-dependency-path: solidarian-id/package-lock.json

- name: Install dependencies
  run: npm ci
```

Código 1: Fase preparatoria en *CI*

Durante dichas etapas se observan las siguientes características:

- **Checkout code**: En esta etapa de descarga del código podemos observar la opción *fetch-depth: 0*, la cual obtiene todo el historial de *git* (necesario para SonarQube luego).

- **Setup Node.js**: En esta etapa de instalar *Node* también se cachean las dependencias para acelerar ejecuciones futuras.
- **Install dependencies**: En esta etapa se descargan las dependencias mediante el comando *npm ci*.

3.2. Fase de formato

Durante esta fase se realizan validaciones de formato con (*Prettier*) y *linting* para asegurar que el código siga los estándares de estilo establecidos por el equipo. Si hay inconsistencias de formato, las corrige automáticamente y hace *commit* de los cambios.
La fase de formato valida y corrige automáticamente el estilo del código en dos niveles: primero, *Prettier* verifica el formato visual (indentación, espacios, comillas) en archivos *TypeScript*, *JavaScript*, *JSON*, *CSS* e *HTML*, y si encuentra inconsistencias, las corrige automáticamente y hace un *commit* con los cambios.
Acto seguido, *ESLint* ejecuta reglas de calidad de código más profundas (detecta *imports* no usados, variables sin usar, prácticas inseguras en *TypeScript*, problemas en *tests*, etc.), con modo estricto que no permite ni advertencias. Todo esto garantiza que el código siga los estándares del equipo antes de compilar y testear.
A continuación se muestran dichas etapas en el trozo de código 2:

```
- name: Run prettier check
  id: prettier_check
  continue-on-error: true
  run: npx prettier --check '**/*.{ts,js,json,css,html}'

- name: Run prettier format
  if: steps.prettier_check.outcome == 'failure'
  run: npx prettier --write '**/*.{ts,js,json,css,html}'

- name: Commit formatted files
  if: steps.prettier_check.outcome == 'failure'
  run: |
    git config --global user.name 'github-actions[bot]'
    git config --global user.email 'github-actions[bot]@users.noreply.github.com'

    git add .
    git commit -m "chore(prettier-formatting): apply prettier formatting [skip ci]" \
      || exit 0

    # Determinar rama destino
    if [ "${{ github.event_name }}" = "pull_request" ]; then
      TARGET_BRANCH="${{ github.head_ref }}"
    else
      TARGET_BRANCH="${{ github.ref_name }}"
    fi

    echo "Pushing to branch: $TARGET_BRANCH"

    # Traer remoto y rebasar para evitar non-fast-forward
    git fetch origin "$TARGET_BRANCH"
    git rebase "origin/$TARGET_BRANCH" || (git rebase --abort && exit 1)

    # Push
    git push origin "HEAD:$TARGET_BRANCH"
```

```yaml
- name: Run linter
  run: npm run lint:ci
```

Código 2: Fase de formato en *CI*

Durante dichas etapas se observan las siguientes características:

- **Run prettier check**: En esta etapa se verifica si el código cumple con el estándar de formato de *Prettier*, verificando todos los archivos cuya extensión aparece en esa lista.
- **Run prettier format**: Si la verificación de la etapa anterior falló, corrige automáticamente el formato de todos los archivos, gracias a la opción `-write`.
- **Commit formatted files**: En esta etapa se configura al *bot* de *GitHub* como autor del *commit*, agrega todos los archivos modificados, crea un *commit* con el mensaje *[skip ci]* para evitar un ciclo infinito, detecta si es una *PR* o *push* directo para saber a qué rama subir, sincroniza con los cambios remotos usando *rebase* (para evitar *merges* innecesarios), y finalmente hace *push* de los cambios formateados a la rama origen.
- **Run linter**: En esta etapa se ejecuta *ESLint* para detectar problemas de calidad más allá del formato.

3.3. Fase de compilación

Esta fase es la más sencilla, ya que simplemente compila el código *TypeScript* a *JavaScript* y prepara todos los artefactos necesarios para ejecutar la aplicación. Durante la compilación nos aseguramos de que no hay errores de tipado, así ya se podrán ejecutar las pruebas con seguridad. Podemos observar dicha etapa en el trozo de código 3.

```yaml
- name: Build project
  run: npm run build:all
```

Código 3: Fase de compilación en *CI*

3.4. Fase de *testing*

Durante esta etapa se ejecuta la *suite* completa de pruebas (unitarias, integración, e2e) del proyecto con reporte de cobertura de código combinado, garantizando que los cambios no rompan funcionalidades existentes y midiendo qué porcentaje del código está siendo probado. El comando `npm run test:all:coverage` por debajo ejecuta los *tests* de todos los microservicios (*users*, *communities* y *admin*) Esta fase del pipeline se puede observar en el trozo de código 4

```yaml
- name: Run all tests with combined coverage
  run: npm run test:all:coverage
```

Código 4: Fase de *testing* en *CI*

3.5. Fase de *SonarQube*

La etapa de *SonarQube* en el *CI* realiza un análisis exhaustivo de calidad del código. Primero, el *pipeline* espera a que *SonarQube* esté completamente operativo (hasta 90 intentos con esperas de 5 segundos), luego crea un proyecto en *SonarQube* de forma idempotente y genera un *token* de autenticación único para cada ejecución del CI. Esto asegura que si el proyecto ya existe, no falla, y que cada análisis tenga credenciales nuevas.

```
- name: Install jq (for JSON parsing)
  run: |
    sudo apt-get update
    sudo apt-get install -y jq

- name: Wait for SonarQube to be UP
  run: |
    for i in {1..90}; do
      STATUS="$(curl -s http://localhost:9000/api/system/status | jq -r '.status' ||
      ↪  true)"
      echo "SonarQube status: $STATUS"
      if [ "$STATUS" = "UP" ]; then
        echo "SonarQube is UP"
        exit 0
      fi
      sleep 5
    done
    echo "SonarQube did not become UP in time"
    docker ps
    docker logs "$(docker ps -q --filter
    ↪  "ancestor=sonarqube:25.12.0.117093-community")" || true
    exit 1

- name: Create project (idempotent) + generate token
  run: |
    PROJECT_KEY="SolidarianID"
    PROJECT_NAME="SolidarianID"

    # Sonarqube API to create project (idempotent)
    curl -s -u admin:admin -X POST "http://localhost:9000/api/projects/create" \
      -d "name=$PROJECT_NAME&project=$PROJECT_KEY" >/dev/null || true

    # Sonarqube API to generate token
    TOKEN_JSON="$(curl -s -u admin:admin -X POST
    ↪  "http://localhost:9000/api/user_tokens/generate" \
      -d "name=ci-${GITHUB_RUN_ID}-${GITHUB_RUN_ATTEMPT}")"

    SONAR_TOKEN="$(echo "$TOKEN_JSON" | jq -r '.token')"

    if [ -z "$SONAR_TOKEN" ] || [ "$SONAR_TOKEN" = "null" ]; then
      echo "Failed to generate SONAR_TOKEN. Response:"
      echo "$TOKEN_JSON"
      exit 1
    fi

    echo "SONAR_TOKEN=$SONAR_TOKEN" >> "$GITHUB_ENV"
    echo "SONAR_PROJECT_KEY=$PROJECT_KEY" >> "$GITHUB_ENV"
    echo "SONAR_PROJECT_NAME=$PROJECT_NAME" >> "$GITHUB_ENV"
```

Una vez que *SonarQube* está listo y autenticado, el *scanner* analiza el código fuente *TypeScript* del proyecto, específicamente los archivos en la carpeta *apps* (microservicios *users*, *communities* y *admin*). El análisis se enfoca en el código fuente (`apps/**/src/**/*.ts`) mientras excluye directorios compilados, dependencias, y archivos de configuración. Simultáneamente, examina los tests (archivos `.spec.ts`, `.test.ts`, y `.e2e-spec.ts`) pero excluye ciertas clases de archivos de la cobertura, como entidades, *DTOs*, *guards*, *mappers* y archivos de configuración que no requieren *testing* completo.

```yaml
- name: Run SonarQube Scanner
  uses: SonarSource/sonarqube-scan-action@a31c9398be7ace6bbfaf30c0bd5d415f843d45e9
  ↪  #SonarSource/sonarqube-scan-action@v7
  env:
    SONAR_TOKEN: ${{ env.SONAR_TOKEN }}
    SONAR_HOST_URL: http://localhost:9000
  with:
    projectBaseDir: solidarian-id
    args: >
      -Dsonar.projectKey=${{ env.SONAR_PROJECT_KEY }}
      -Dsonar.projectName=${{ env.SONAR_PROJECT_NAME }}
      -Dsonar.sources=apps
      -Dsonar.inclusions=apps/**/src/**/*.ts
      -Dsonar.exclusions=**/dist/**,**/node_modules/**,**/coverage/**,**/*.spec.ts,**
      ↪  /*.test.ts,**/*.e2e-spec.ts,**/*.js,**/*.json,**/*.css,**/*.html
      -Dsonar.tests=apps
      -Dsonar.test.inclusions=**/*.spec.ts,**/*.test.ts,**/*.e2e-spec.ts
      -Dsonar.coverage.exclusions=**/*.entity.ts,**/*.dto.ts,**/*.filter.ts,**/*.guar
      ↪  d.ts,**/*.mapper.ts,**/*.port.ts,**/fake*.ts,**/enums/**,**/model/**,**/mod
      ↪  els/**,**/main.ts,**/*.module.ts,**/jest*.config.ts,**/*.config.ts,**/*.d.t
      ↪  s,**/index.ts
      -Dsonar.javascript.lcov.reportPaths=coverage/lcov.info
      -Dsonar.qualitygate.wait=true
```

Finalmente, *SonarQube* valida que el proyecto cumple con la *Quality Gate* definida, bloqueando el *pipeline* si se detectan problemas graves. El *pipeline* también descarga y genera un reporte *CNES* (del portal *CNES* de auditoría) que proporciona una visión más detallada de la calidad y lo carga como artefacto en *GitHub Actions* para que pueda ser descargado y revisado por los desarrolladores.

```yaml
- name: Setup Java (required by CNES)
  uses: actions/setup-java@f2beeb24e141e01a676f977032f5a29d81c9e27e
  ↪  #actions/setup-java@v5.1.0
  with:
    distribution: temurin
    java-version: "17"

- name: Download CNES Report (standalone jar)
  run: |
    mkdir -p reports
    curl -L -o reports/sonar-cnes-report.jar \
      https://github.com/cnescatlab/sonar-cnes-report/releases/download/5.0.2/sonar-c
      ↪  nes-report-5.0.2.jar

- name: Generate CNES report
  run: |
```

```
    mkdir -p reports/sonar
    java -jar reports/sonar-cnes-report.jar \
      -s http://localhost:9000 \
      -t "${{ env.SONAR_TOKEN }}" \
      -p "${{ env.SONAR_PROJECT_KEY }}" \
      -o reports/sonar

- name: Upload CNES report
  uses: actions/upload-artifact@b7c566a772e6b6bfb58ed0dc250532a479d7789f
  with:
    name: sonar-cnes-report
    path: solidarian-id/reports/sonar
```

Código 5: Fase de *SonarQube* en *CI*

Durante dichas etapas se observan las siguientes características:

- **Install jq (for JSON parsing)**: Se instala *jq* debido a que en las siguientes etapas se necesita para manipular respuestas en formato *JSON*.
- **Wait for SonarQube to be UP**: Esta etapa realiza un *health check* del servicio *SonarQube* mediante *polling*. Intenta hasta 90 veces (con intervalos de 5 segundos) consultar el *endpoint* `/api/system/status` para verificar que *SonarQube* está completamente operativo. Si el servicio no responde en ese tiempo, muestra *logs* de *Docker* para *debugging* y falla el *pipeline*. Esto es crítico porque las siguientes etapas dependen de que *SonarQube* esté disponible.
- **Create project (idempotent) + generate token**: El proyecto en *SonarQube* se crea usando las credenciales por defecto `admin:admin`) de forma idempotente. El *token* se extrae del *JSON* de respuesta usando *jq* y se valida que no sea vacío antes de continuar. Este *token* se usa para recopilar los datos del reporte.
- **Run SonarQube Scanner**: Es el corazón del análisis. Utiliza la acción `SonarSource/sonarqube-scan-action` que ejecuta el *scanner*. Activa `qualitygate.wait=true` para bloquear el *pipeline* si no se cumple la calidad definida.
- **Setup Java + Download CNES Report + Generate CNES report**: Estas tres subetapas generan un reporte complementario. Instalan *Java 17* (requisito de la herramienta *CNES*), descargan el *jar* ejecutable de *sonar-cnes-report*, y lo ejecutan contra *SonarQube* con el *token* generado para producir un reporte detallado en formato `HTML/PDF`. Este reporte se carga como artefacto de *GitHub Actions* para que los desarrolladores puedan descargarlo y revisarlo en detalle.

4. Diseño de un pipeline de despliegue

El *pipeline* de *CD* (*Continuous Deployment*) de *SolidarianID* se ejecuta automáticamente cuando se realiza un *push* a la rama *main* (después de que el código ha pasado el *CI* en *develop* y ha sido *mergeado* a *main*). Este *pipeline* es responsable de construir y distribuir las imágenes *Docker* de las tres aplicaciones (*admin*, *users* y *communities*), así como de orquestar la infraestructura en *AWS* y desplegar la aplicación en producción.
El *pipeline* se divide en varias fases:

1. **Preparación del código**: Esta etapa descarga el código del repositorio y genera un *tag* único para las imágenes *Docker* combinando el *hash* corto del *commit* actual

con un *timestamp*. Además, configura la autenticación en *Docker Hub* para poder *pushear* las imágenes posteriormente.

2. **Construcción y distribución de *Docker***: En esta etapa se construyen las tres imágenes *Docker* (*admin*, *users* y *communities*) utilizando el *docker-compose*, luego se etiquetan con el *tag* generado en la etapa anterior y la versión *latest*, y finalmente se *pushean* a *Docker Hub*. De esta forma, las imágenes quedan disponibles en el registro para ser descargadas en producción de el docker-compose que las referencie.
3. **Despliegue en *AWS***: Esta es la etapa final donde se configura la autenticación en *AWS*, se despliegan los *stacks* de *CloudFormation* (primero la red y luego la aplicación), se fuerza la redescarga y recreación de los contenedores en la instancia *EC2* mediante *AWS Systems Manager* para asegurar que se usen las imágenes más recientes, y finalmente se obtienen y muestran las *URLs* de acceso de las tres aplicaciones.

No obstante, antes de afrontar el fichero de configuración del *pipeline*, vamos a ver los dos ficheros de definición de los *stacks* que vamos a desplegar, para saber qué estamos desplegando antes de ver cómo lo hacemos.

4.1. Stacks desplegados

En esta subsección trataremos los dos ficheros de *stack* que vamos a desplegar:

- `lab-network.yaml`: Contiene todo lo referente a utilidades de red para poder acceder a nuestro *stack principal* desde internet.
- `stack.yaml`: Contiene lo necesario para ejecutar nuestras 3 aplicaciones, Kafka y las bases de datos correspondientes.

4.1.1. lab-network.yaml

Este fichero construye la topología de red necesaria para que el entorno sea funcional y accesible desde el exterior. Podemos dividir su funcionamiento en tres bloques lógicos:

1. **Definición de la Red (VPC)**: Se crea una *Virtual Private Cloud* con el rango `10.0.0.0/16`. Se habilitan explícitamente la resolución y los nombres de host DNS, lo cual es indispensable para que los contenedores y servicios puedan descubrirse entre sí utilizando nombres de dominio internos en lugar de IPs volátiles.
2. **Acceso Público y Enrutamiento**: Para que la red no quede aislada, se despliega un *Internet Gateway* y se conecta a la VPC. Posteriormente, se configura una tabla de enrutamiento (*PublicRouteTable*) que define una "ruta por defecto" (`0.0.0.0/0`) hacia este Gateway. Esto permite que cualquier recurso asociado a esta tabla tenga salida directa a internet.
3. **Subnet y Exportación**: Finalmente, se define una subred pública (`10.0.0.0/24`) en una zona de disponibilidad específica y se asocia a la tabla de enrutamiento anterior. El punto clave de este *stack* es su sección `Outputs`: exporta el `VpcId` y el `SubnetId` con nombres dinámicos. Esto actúa como un "contrato" de interfaz, permitiendo que el siguiente *stack* (la aplicación) consuma estos recursos de red sin necesidad de conocer sus IDs físicos.

```
AWSTemplateFormatVersion: 2010-09-09
Description: SolidarianID Lab Network

Resources:
  VPC:
```

```
    Type: AWS::EC2::VPC
    Properties:
      EnableDnsSupport: true
      EnableDnsHostnames: true
      CidrBlock: 10.0.0.0/16

  InternetGateway:
    Type: AWS::EC2::InternetGateway

  VPCGatewayAttachment:
    Type: AWS::EC2::VPCGatewayAttachment
    Properties:
      VpcId: !Ref VPC
      InternetGatewayId: !Ref InternetGateway

  PublicRouteTable:
    Type: AWS::EC2::RouteTable
    Properties:
      VpcId: !Ref VPC

  PublicRoute:
    Type: AWS::EC2::Route
    DependsOn: VPCGatewayAttachment
    Properties:
      RouteTableId: !Ref PublicRouteTable
      DestinationCidrBlock: 0.0.0.0/0
      GatewayId: !Ref InternetGateway

  PublicSubnet:
    Type: AWS::EC2::Subnet
    Properties:
      VpcId: !Ref VPC
      CidrBlock: 10.0.0.0/24
      AvailabilityZone: !Select
        - 0
        - !GetAZs
          Ref: AWS::Region

  PublicSubnetRouteTableAssociation:
    Type: AWS::EC2::SubnetRouteTableAssociation
    Properties:
      SubnetId: !Ref PublicSubnet
      RouteTableId: !Ref PublicRouteTable

  PublicSubnetNetworkAclAssociation:
    Type: AWS::EC2::SubnetNetworkAclAssociation
    Properties:
      SubnetId: !Ref PublicSubnet
      NetworkAclId: !GetAtt
        - VPC
        - DefaultNetworkAcl

Outputs:
  PublicSubnet:
    Description: The subnet ID to use for public web servers
    Value: !Ref PublicSubnet
    Export:
      Name: !Sub "${AWS::StackName}-SubnetID"
```

```
  VPC:
    Description: VPC ID
    Value: !Ref VPC
    Export:
      Name: !Sub "${AWS::StackName}-VPCID"
```

Código 6: lab-network.yaml como está en el repositorio de la práctica

4.1.2. stack.yaml

Este fichero es el núcleo de nuestra infraestructura. Define la instancia EC2, su configuración de seguridad, los permisos de IAM y el proceso de aprovisionamiento automático (User Data) para desplegar los contenedores.

Parámetros de Entrada El stack comienza definiendo parámetros que permiten personalizar el despliegue sin modificar el código fuente. Destacan NetworkStackName para vincularse con la red creada previamente y DockerHubUsername para descargar las imágenes correctas. La imagen hemos puesto una versión de Amazon Linux fija porque si pusieramos la versión *latest* si cambia el puntero a una nueva versión nos podria ocasionar problemas con la instancia en los redespliegues. Como hacemos uso de un proxy inverso para redirigir a nuestra pagina web `soliarianid.com` las peticiones (se explicará en la sección Dominio de SolidarianID) hemos optado por crear una ip elástica previamente y asociar su *eipalloc-id* en el fichero.

```
Parameters:
  NetworkStackName:
    Description: Name of the networking stack
  Type: String
  Default: MyStackNetwork

  AmazonLinuxAMIID:
    Type: AWS::EC2::Image::Id
    Default: ami-02e50d15ca7664be9

  DockerHubUsername:
    Description: Docker Hub username for pulling images
    Type: String
    Default: zerep0

  ExistingEipAllocationId:
    Type: String
    Default: eipalloc-086485758b47098be
```

Código 7: Parámetros de entrada en stack.yaml

Definición de la Instancia EC2 El recurso principal es la instancia WebServerInstance. Se configura como una máquina tipo t3.medium y se le asigna la subred importada del stack de red usando Fn::ImportValue. Además, se utiliza Metadata con AWS::CloudFormation::Init para preinstalar paquetes como Docker.

```
Resources:
```

```yaml
  WebServerInstance:
    Type: AWS::EC2::Instance
    Metadata:
      "AWS::CloudFormation::Init":
        configSets:
          All:
            - ConfigureSampleApp
        ConfigureSampleApp:
          packages:
            yum:
              docker: []
            services:
              sysvinit:
                docker:
                  enabled: true
                  ensureRunning: true
      Properties:
        InstanceType: t3.medium
        ImageId: !Ref AmazonLinuxAMIID
        SubnetId:
          Fn::ImportValue: !Sub ${NetworkStackName}-SubnetID
```

Código 8: Configuración base de la instancia EC2

Script de Aprovisionamiento (User Data) Esta sección contiene el script Bash que se ejecutará al arrancar la instancia. Realiza tareas críticas como instalar docker-compose, descargar las credenciales de producción desde AWS Secrets Manager [2] y clonar los datos iniciales desde un bucket S3 [3]. Además, generará un docker-compose dinámico que contendrá las versiones más recientes de las imagenes *pusheadas* a Docker Hub como se explicará a continuación.

```yaml
UserData:
Fn::Base64: !Sub |
#!/bin/bash -e
# Instalamos docker-compose
sudo curl -L "https://github.com/docker/compose/..." -o /usr/local/bin/docker-compose
# Obtenemos las credenciales desde AWS Secrets Manager
SECRET_JSON=$(aws secretsmanager get-secret-value \
  --secret-id "prod/solidarianid" \
  --query SecretString \
  --output text \
  --region ${AWS::Region})

# Generamos el archivo .env
echo "$SECRET_JSON" | jq -r 'to_entries | ...' > /home/ec2-user/solidarian.env

# Descargamos datos iniciales de S3
aws s3 cp s3://solidarianid-bucket-init-data-database /home/ec2-user/init-data ...
```

Código 9: User Data para la configuración automática

Seguridad y Permisos (IAM Role) Para que la instancia pueda acceder a Secrets Manager y S3 sin quemar credenciales en el código, se crea un rol de IAM específico

(WebServerInstanceRole). Este rol sigue el principio de mínimo privilegio, otorgando acceso solo a los secretos y buckets necesarios.

```
WebServerInstanceRole:
    Type: AWS::IAM::Role
    Properties:
        AssumeRolePolicyDocument:
        # ... (Trust policy para ec2.amazonaws.com)
        Policies:
            - PolicyName: ReadSolidarianSecrets
              PolicyDocument:
                Statement:
                  - Effect: Allow
                    Action: secretsmanager:GetSecretValue
                    Resource: !Sub arn:aws:secretsmanager:${AWS::Region}:${AWS::Accou
                    ↪  ntId}:secret:prod/solidarianid*
            - PolicyName: S3ReadAccess
              PolicyDocument:
                Statement:
                  - Effect: Allow
                    Action: s3:GetObject
                    Resource: !Sub
                    ↪  arn:aws:s3:::solidarianid-bucket-init-data-database/*
```

Código 10: Rol de IAM con permisos granulares

Grupo de Seguridad y Red Se configura un Security Group que actúa como firewall virtual. Permite el tráfico SSH (puerto 22) para administración y abre los puertos de los microservicios (3000-3002) para el acceso público. Además, se asocia la IP elástica existente mencionada antes.

```
WebServerSecurityGroup:
    Type: AWS::EC2::SecurityGroup
    Properties:
        GroupDescription: Security group for SolidarianID microservices
        VpcId:
            Fn::ImportValue: !Sub ${NetworkStackName}-VPCID
        SecurityGroupIngress:
            - IpProtocol: tcp
              FromPort: 22
              ToPort: 22
              CidrIp: 0.0.0.0/0
            - IpProtocol: tcp
              FromPort: 3000
              ToPort: 3002
              CidrIp: 0.0.0.0/0

WebServerEIPAssociation:
    Type: AWS::EC2::EIPAssociation
    Properties:
        AllocationId: !Ref ExistingEipAllocationId
        InstanceId: !Ref WebServerInstance
```

Código 11: Configuración de red y seguridad

Outputs Finalmente, el stack exporta las URLs completas de los tres microservicios utilizando el nombre DNS público de la instancia. Estos valores son los que recoge el pipeline de GitHub Actions para validar el despliegue.

```
Outputs:
    AdminURL:
        Description: URL of the admin application
        Value: !Sub "http://${WebServerInstance.PublicDnsName}:3000"
    UsersURL:
        Description: URL of the users microservice
        Value: !Sub "http://${WebServerInstance.PublicDnsName}:3001"
    CommunitiesURL:
        Description: URL of the communities microservice
        Value: !Sub "http://${WebServerInstance.PublicDnsName}:3002"
```

Código 12: Outputs con las URLs de los servicios

4.2. Fase de preparación

La primera etapa prepara todo lo necesario: obtiene el código, genera un identificador único para la versión que se va a desplegar, y configura la autenticación en *Docker Hub* para poder distribuir las imágenes posteriormente. Obsérvese el trozo de código 13.

```
- name: Checkout code
  uses: actions/checkout@v4
  with:
    fetch-depth: 0

- name: Generate image tag
  id: image-tag
  run: |
    SHORT_SHA=$(echo ${{ github.sha }} | cut -c1-7)
    IMAGE_TAG="${SHORT_SHA}-$(date +%s)"
    echo "tag=${IMAGE_TAG}" >> $GITHUB_OUTPUT

- name: Login to Docker Hub
  uses: docker/login-action@v3
  with:
    username: ${{ secrets.DOCKERHUB_USERNAME }}
    password: ${{ secrets.DOCKERHUB_ACCESS_TOKEN }}
```

Código 13: Fase de preparación en *CD*

Durante dichas etapas se observan las siguientes características:

- **Checkout code**: En esta etapa de descarga del código podemos observar la opción *fetch-depth: 0*, la cual obtiene todo el historial de *git* incluidos los nuevos cambios.
- **Generate image tag**: Este paso genera un identificador único para las imágenes *Docker* que se van a crear. Primero extrae los primeros 7 caracteres del *hash* del *commit* actual (`SHORT_SHA`), que identifica de forma única el código en ese momento. Luego lo combina con un *timestamp* (fecha y hora actual en segundos), creando así un *tag* como `a1b2c3d-1735987654`. Este *tag* único garantiza que cada despliegue pueda ser rastreado e identificado, y permite saber exactamente qué versión del código está ejecutándose en producción.

- **Login to Docker Hub**: Este paso autentica el *runner* (la máquina virtual que ejecuta el *workflow*) en *Docker Hub* utilizando las credenciales almacenadas en los secretos de *GitHub* (`DOCKERHUB_USERNAME` y `DOCKERHUB_ACCESS_TOKEN`). Esta autenticación es necesaria para poder *pushear* (subir) las imágenes *Docker* al repositorio en la siguiente etapa.

4.3. Fase de *Docker*

Este es el paso más crítico de la etapa de dockerización. Podemos observar dicha etapa en el trozo de código 16.
Comienza construyendo las tres imágenes *Docker* (*admin-app*, *user-app* y *community-app*) usando el comando `docker compose -p deployment build`, fijando *deployment* como nombre del proyecto. Luego estas seran pusheadas a DockerHub para su posterior uso. A continuación se ilustrará con un ejemplo la elaboración de las imagenes de cada app y posteriormente como se enviaron a DockerHub mediante el pipeline.

4.3.1. Construcción de la imagen Docker

Para el despliegue del monolito de administración se ha definido una imagen Docker basada en *multi-stage build*, con el objetivo de separar el proceso de compilación (más pesado y con dependencias de desarrollo) de la imagen final de ejecución (más ligera y segura). En primer lugar, se utiliza una etapa `builder` que instala dependencias, genera los recursos estáticos (Tailwind) y compila la aplicación `admin`.

```
FROM node:24-alpine AS builder
WORKDIR /app

COPY package*.json ./
RUN npm ci && npm cache clean --force

COPY . ./
RUN npm run build:tailwind
RUN npm run build -- admin
```

Código 14: Etapa `builder` para compilar la aplicación `admin`

Posteriormente, una etapa `production` instala únicamente dependencias de producción y copia exclusivamente los artefactos necesarios para ejecutar el servicio, incorporando además un usuario no privilegiado y un *healthcheck* para facilitar la supervisión en ejecución. De forma análoga se han realizado el resto de aplicaciones sin contar con tailwind debido a que las otras no están orientadas actualmente a frontend.

```
FROM node:24-alpine AS production
WORKDIR /app

RUN apk add --no-cache dumb-init \
  && addgroup -g 1001 -S nodejs \
  && adduser -S nestjs -u 1001

COPY package*.json ./
RUN npm ci --omit=dev && npm cache clean --force

COPY --from=builder --chown=nestjs:nodejs /app/dist/apps/admin ./dist/apps/admin
COPY --from=builder --chown=nestjs:nodejs /app/apps/admin/views ./apps/admin/views
COPY --from=builder --chown=nestjs:nodejs /app/apps/admin/public ./apps/admin/public
COPY --from=builder --chown=nestjs:nodejs
↪  /app/libs/design-tokens/src/icons/solidarian-128x128.png
↪  ./libs/design-tokens/src/icons/solidarian-128x128.png

EXPOSE 3000

HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \
  CMD node -e "require('http').get('http://127.0.0.1:3000/health',
  ↪  r=>process.exit(r.statusCode===200?0:1)).on('error',()=>process.exit(1))"

USER nestjs
ENTRYPOINT ["dumb-init", "--"]
CMD ["node", "dist/apps/admin/main.js"]
```

Código 15: Etapa `production` para ejecutar la aplicación `admin`

4.3.2. Envio de las imágenes a DockerHub

Una vez construidas, cada aplicación se etiqueta de dos formas diferentes. Primero, se le asigna el *tag* único generado anteriormente, que permite identificar la versión del código de la imagen. Segundo, se le asigna el *tag latest*, que siempre apunta a la versión más reciente y es útil para *deployments* que simplemente quieren la última versión sin especificar un *tag* exacto.
Después del etiquetado, cada imagen se *pushea* (sube) a *Docker Hub* en dos ocasiones: una con el *tag* específico y otra con el *tag latest*. Esto se hace para las tres aplicaciones (*admin*, *users* y *communities*), asegurando que todas estén disponibles en el repositorio de *Docker Hub* con versiones identificables y una referencia *latest* siempre actualizada. De esta forma, cualquier instancia en producción puede descargar estas imágenes cuando sea necesario.

```
- name: Build and push Docker images
  id: build-images
  working-directory: deployment
  env:
    IMAGE_TAG: ${{ steps.image-tag.outputs.tag }}
  run: |
    set -euo pipefail

    # Build all 3 apps
    docker compose -p deployment build admin-app user-app community-app

    # Tag and push admin-app
```

```
docker tag deployment-admin-app:latest ${{ secrets.DOCKERHUB_USERNAME
↪  }}/solidarianid-admin:$IMAGE_TAG
docker tag deployment-admin-app:latest ${{ secrets.DOCKERHUB_USERNAME
↪  }}/solidarianid-admin:latest
docker push ${{ secrets.DOCKERHUB_USERNAME }}/solidarianid-admin:$IMAGE_TAG
docker push ${{ secrets.DOCKERHUB_USERNAME }}/solidarianid-admin:latest

# Tag and push users-app
docker tag deployment-user-app:latest ${{ secrets.DOCKERHUB_USERNAME
↪  }}/solidarianid-user:$IMAGE_TAG
docker tag deployment-user-app:latest ${{ secrets.DOCKERHUB_USERNAME
↪  }}/solidarianid-user:latest
docker push ${{ secrets.DOCKERHUB_USERNAME }}/solidarianid-user:$IMAGE_TAG
docker push ${{ secrets.DOCKERHUB_USERNAME }}/solidarianid-user:latest

# Tag and push communities-app
docker tag deployment-community-app:latest ${{ secrets.DOCKERHUB_USERNAME
↪  }}/solidarianid-community:$IMAGE_TAG
docker tag deployment-community-app:latest ${{ secrets.DOCKERHUB_USERNAME
↪  }}/solidarianid-community:latest
docker push ${{ secrets.DOCKERHUB_USERNAME }}/solidarianid-community:$IMAGE_TAG
docker push ${{ secrets.DOCKERHUB_USERNAME }}/solidarianid-community:latest
```

Código 16: Fase de *Docker* en *CD*

4.4. Fase de despliegue en AWS

A continuación, se presenta una visión general de los principales aspectos de la arquitectura del sistema mediante el esquema mostrado en la Figura 1.

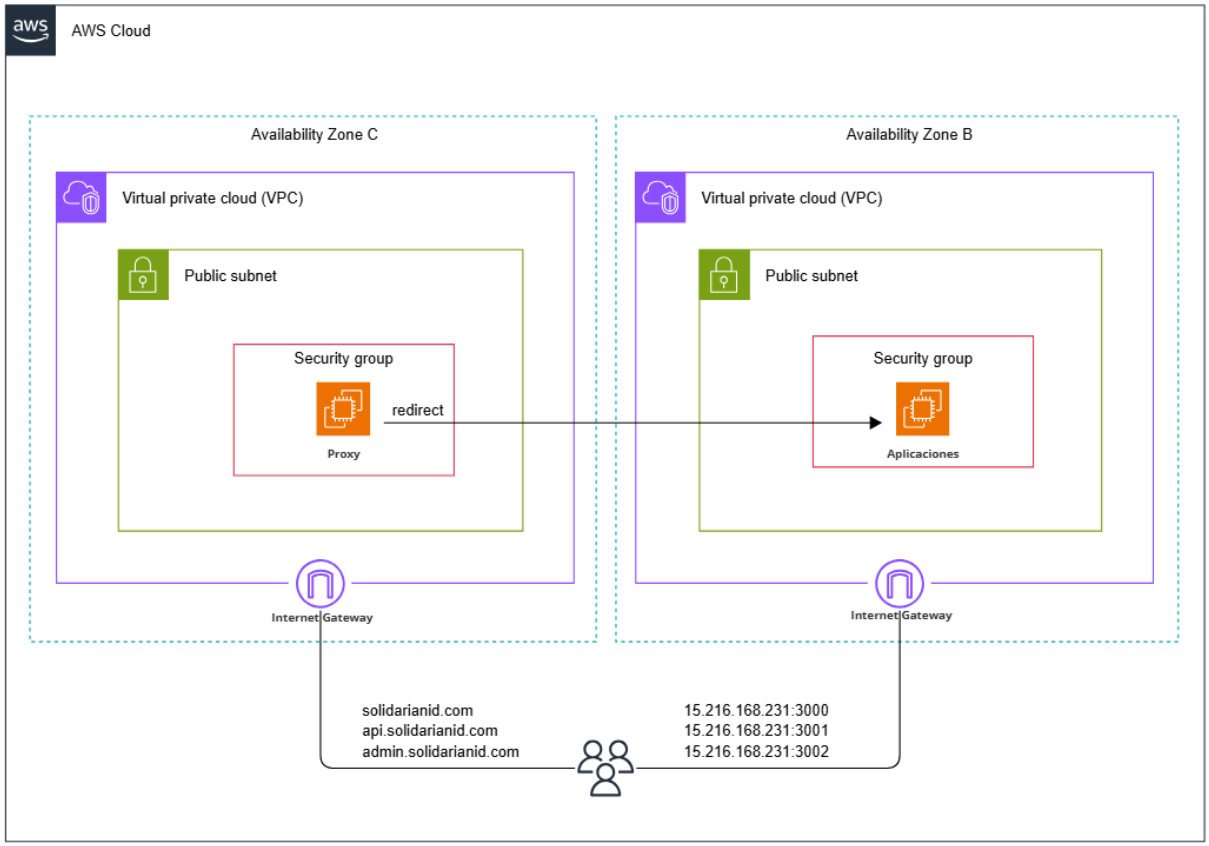


Figura 1: Arquitectura de despliegue en AWS

Tal y como se observa en la Figura 1, se ha desplegado una instancia *EC2* de tipo `t3.nano` dedicada a actuar como *reverse proxy* mediante **Nginx**. Esta instancia es el punto de entrada público del sistema, exponiendo el tráfico HTTPS hacia Internet y redirigiendo las peticiones entrantes a las aplicaciones desplegadas en la infraestructura interna.
Para habilitar el acceso mediante nombre de dominio, se adquirió el dominio en IONOS (`solidarianid.com`) y se configuraron los registros *DNS* para que apunten a la instancia *proxy*. En la configuración de Nginx, ubicada en `/etc/nginx/sites-available/solidarianid-proxy.conf` dentro de la instancia, se definieron las reglas de enrutamiento necesarias para permitir el uso de subdominios asociados al dominio principal (véase Listado 17).

```
server {
  listen 443 ssl;
  server_name admin.solidarianid.com;

  ssl_certificate     /etc/nginx/ssl/cert.cer;
  ssl_certificate_key /etc/nginx/ssl/privkey.key;
  ssl_protocols TLSv1.2 TLSv1.3;

  location / {
    proxy_pass http://15.216.168.231:3000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }
}

server {
  listen 443 ssl;
  server_name api.solidarianid.com;
  ...
  location /users {
    proxy_pass http://15.216.168.231:3001/api/users;
    ...
  }

  location /communities/ {
    proxy_pass http://15.216.168.231:3002/api/communities;
    ...
  }

  location /causes/ {
    proxy_pass http://15.216.168.231:3002/api/causes;
    ...
  }

  location /actions/ {
    proxy_pass http://15.216.168.231:3002/api/actions;
    ...
  }

  location /requests/ {
    proxy_pass http://15.216.168.231:3002/api/;
    ...
  }
}
```

Código 17: Fragmento reducido de `solidarianid-proxy.conf` (Nginx reverse proxy)

La estructura planteadasigue el principio de que el *DNS apunte siempre al proxy*, siendo este el encargado de redirigir y enrutar las peticiones hacia las rutas y puertos internos correspondientes. En particular:

- El subdominio `admin.solidarianid.com` se utiliza para acceder al monolito de administración, que incluye la interfaz web.
- El subdominio `api.solidarianid.com` se emplea como punto de acceso unificado para las **APIs** de los distintos microservicios.
- La instancia *proxy* dispone de una IP elástica asociada, proporcionando un punto

de entrada fijo y estable, independiente de reinicios o recreaciones de la instancia.
- Nginx recibe tráfico HTTPS en el puerto 443 y lo reenvía internamente a los servicios desplegados en la instancia de aplicaciones, identificados mediante `IP:puerto`.

Adicionalmente, los servicios también pueden ser consultados de forma directa mediante `IP:puerto` (por ejemplo, admin es `15.216.168.231:3000`, usuarios es `15.216.168.231:3001` y comunidades es `15.216.168.231:3002`). Aunque visualmente solo se muestra admin ya que no se contempla el resto para la actual entrega. Los otros proyectos solo proporcionan sus consecuentes APIs.
Con respecto a la segunda instancia de AWS (encargada de ejecutar las aplicaciones), y dado que su configuración abarca múltiples elementos (red, despliegue, dependencias y operación), se ha optado por estructurar su descripción en tres subapartados para mejorar la claridad y la trazabilidad de la explicación. En concreto, cada subapartado se centra en un objetivo bien delimitado:

1. **Configuración del acceso externo:** se detalla cómo se habilita la configuración de red seleccionada para nuestro stack.
2. **Despliegue de la infraestructura:** se describe la provisión y puesta en marcha del entorno de ejecución, incluyendo la instalación y configuración de los componentes necesarios (por ejemplo, *runtime*, contenedores y servicios auxiliares), así como la preparación del sistema para ejecutar los microservicios.
3. **Actualización y validación en ejecución:** se explica el procedimiento seguido para publicar nuevas versiones, aplicar cambios sin comprometer la disponibilidad, y verificar que el sistema queda operativo tras el despliegue.

Esta separación permite presentar el proceso de forma progresiva (acceso → infraestructura → operación), evitando mezclar decisiones de red con detalles de despliegue y facilitando la revisión del flujo completo de extremo a extremo.

4.4.1. Configuración de credenciales

Durante este paso se configuran las credenciales de *AWS*. Podemos observar dicha etapa en el trozo de código 18.
La etapa de *Configure AWS credentials* es fundamental para que el *runner* de *GitHub Actions* pueda comunicarse con *AWS* y ejecutar operaciones en la cuenta configurada. Para ello, utiliza la acción `aws-actions/configure-aws-credentials@v4`.
Lo que hace internamente es tomar las tres credenciales almacenadas en los secretos de *GitHub* y configurarlas como variables de entorno en el *runner*:

- **aws-access-key-id**: Es la clave de acceso pública del usuario de *IAM* en *AWS*. Identifica quién eres en *AWS* (en este caso el compañero **Antonio Pérez**), similar a un nombre de usuario.
- **aws-secret-access-key**: Es la clave secreta privada asociada a esa clave de acceso. Actúa como una contraseña y nunca debe ser expuesta públicamente. Por eso se almacena como secreto en *GitHub*.
- **aws-region**: Define la región de *AWS*. Nosotros usamos *eu-south-2* (la región de España) ya que estamos desplegando en el laboratorio personal de uno de los compañeros del grupo (Antonio) en lugar del de prácticas de PC.

Una vez configuradas estas credenciales, el *runner* puede ejecutar comandos de *AWS CLI* sin necesidad de autenticación adicional. Estas credenciales permanecen activas durante toda la ejecución del *workflow*, pero se limpian automáticamente cuando el *workflow* termina por razones de seguridad.

```yaml
- name: Configure AWS credentials
  uses: aws-actions/configure-aws-credentials@v4
  with:
    aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
    aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
    aws-region: eu-south-2
```

Código 18: Fase de configuración de credenciales de *AWS*

4.4.2. Despliegue de infraestructura

Durante este paso se establece toda la infraestructura en *AWS* necesaria para ejecutar las aplicaciones de forma escalable, disponible y segura. Podemos observar dicha etapa en el trozo de código 19. Usaremos para ello los ficheros de *stacks* que hemos visto previamente. Durante la etapa *Deploy Network Stack* se despliega el primer *stack* de *CloudFormation* llamado *MyStackNetwork* usando el fichero *lab-network.yaml*.
El parámetro `-capabilities CAPABILITY_NAMED_IAM CAPABILITY_AUTO_EXPAND` autoriza a *CloudFormation* a crear recursos de *IAM (Identity and Access Management)* que puedan ser nombrados explícitamente, lo cual es necesario para *security groups* y roles. El `-no-fail-on-empty-changeset` evita que el comando falle si el *stack* ya existe y no hay cambios pendientes.
Posteriormente, durante la etapa *Deploy Application Stack* se despliega el segundo *stack* llamado *MyStackApp* utilizando el archivo *stack.yaml*.
Los parámetros que indicamos tienen las siguientes funcionalidades:

- El parámetro `DockerHubUsername=${{ secrets.DOCKERHUB_USERNAME }}` pasa el nombre de usuario de *Docker Hub* al *template* de *CloudFormation*, para que las instancias sepan de dónde descargar las imágenes de *Docker*.
- `AmazonLinuxAMIID` indica el identificador exacto de la versión del SO a usar por las máquinas EC2.
- `ExistingEipAllocationId` es un ID de una IP elástica que reservamos previamente en el tenant de AWS.
- `NetworkStackName` indica que queremos usar el *stack* de red desplegado en el paso previo.
- Los dos últimos flags indican que CloudFormation tiene permisos para crear recursos de IAM con nombres personalizados (`CAPABILITY_NAMED_IAM`) y que el *pipeline* no falle si al ejecutar el comando no hay ningún cambio real en la infraestructura (`-no-fail-on-empty-chest`).

```yaml
- name: Deploy Network Stack
  working-directory: solidarian-id
  run: |
    aws cloudformation deploy \
      --stack-name MyStackNetwork \
      --template-file lab-network.yaml \
      --capabilities CAPABILITY_NAMED_IAM CAPABILITY_AUTO_EXPAND \
      --no-fail-on-empty-changeset

- name: Deploy Application Stack
  working-directory: solidarian-id
  run: |
    aws cloudformation deploy \
```

```
      --stack-name MyStackApp \
      --template-file stack.yaml \
      --parameter-overrides \
        DockerHubUsername=${{ secrets.DOCKERHUB_USERNAME }} \
        AmazonLinuxAMIID=ami-02e50d15ca7664be9 \
        ExistingEipAllocationId=eipalloc-086485758b47098be \
        NetworkStackName=MyStackNetwork \
      --capabilities CAPABILITY_NAMED_IAM \
      --no-fail-on-empty-changeset
```

Código 19: Fase de despliegue en *AWS*

4.4.3. Actualización de contenedores y validación

Durante este paso se ejecutan los contenedores *Docker* más recientes en la instancia *EC2*, se obtienen las *URLs* de acceso y se notifica el resultado. Podemos observar dicha etapa en el trozo de código 20.

La etapa *Force new deployment (if stack unchanged)* garantiza que se ejecuten las versiones más recientes de las aplicaciones, incluso si la infraestructura no ha cambiado. Los pasos detallados son:

1. Primero, obtiene el ID físico de la instancia *EC2* mediante el comando *aws cloudformation describe-stack-resources*, buscando el recurso lógico *WebServerInstance* dentro del stack *MyStackApp*. Si no encuentra un ID válido, el *script* detiene la ejecución inmediatamente.
2. Una vez identificado el servidor, utiliza *AWS Systems Manager* (SSM) [4] para enviar comandos remotos. La instrucción enviada es: `cd /home/ec2-user && docker-compose -env-file /home/ec2-user/.env.solidarianid pull && docker-compose -env-file /home/ec2-user/.env.solidarianid up -d -force-recreate`.
   A diferencia de un despliegue estándar, aquí se especifica explícitamente el archivo de entorno `.env.solidarianid` tanto para la descarga (*pull*) como para el levantamiento de los servicios. El parámetro `-force-recreate` asegura que los contenedores se destruyan y recreen desde cero para aplicar la nueva imagen descargada.
3. Finalmente, el *script* espera la confirmación del comando mediante `aws ssm wait command-executed`. Tras finalizar, verifica si el estado devuelto es `Success`. En caso contrario, recupera la invocación del comando para mostrar los detalles del error en los *logs* y termina el flujo con error.

Posteriormente, en la etapa *Get deployment URLs*, se consulta el stack de *CloudFormation* para recuperar los *Outputs* exportados. Finalmente se extrae las direcciones de *AdminURL*, *UsersURL* y *CommunitiesURL*, escribiéndolas en el *GITHUB_OUTPUT* para hacerlas disponibles en pasos subsiguientes.

Las etapas *Deployment Status* y *Deployment Failure Notification* gestionan la comunicación final. Si el flujo fue exitoso (`if: $ success()`), se imprimen en consola las tres *URLs* de acceso para el usuario. Si hubo algún fallo previo (`if: $ failure()`), se emite una notificación de error simple alertando que el despliegue no se completó correctamente.

```
- name: Force new deployment (if stack unchanged)
  run: |
    set -euo pipefail
```

```
    INSTANCE_ID="$(aws cloudformation describe-stack-resources \
      --stack-name MyStackApp \
      --logical-resource-id WebServerInstance \
      --query 'StackResources[0].PhysicalResourceId' \
      --output text)"

    if [ -z "$INSTANCE_ID" ] || [ "$INSTANCE_ID" = "None" ]; then
      echo "No instance id found"
      exit 1
    fi

    echo "Forcing deployment on instance: $INSTANCE_ID"

    CMD_ID="$(aws ssm send-command \
      --instance-ids "$INSTANCE_ID" \
      --document-name "AWS-RunShellScript" \
      --parameters 'commands=["cd /home/ec2-user && docker-compose --env-file
      ↪  /home/ec2-user/solidarian.env pull && docker-compose --env-file
      ↪  /home/ec2-user/solidarian.env up -d --force-recreate"]' \
      --query "Command.CommandId" \
      --output text)"

    aws ssm wait command-executed --command-id "$CMD_ID" --instance-id "$INSTANCE_ID"

    STATUS="$(aws ssm get-command-invocation \
      --command-id "$CMD_ID" \
      --instance-id "$INSTANCE_ID" \
      --query "Status" \
      --output text)"

    if [ "$STATUS" != "Success" ]; then
      echo "SSM failed: $STATUS"
      aws ssm get-command-invocation --command-id "$CMD_ID" --instance-id
      ↪  "$INSTANCE_ID"
      exit 1
    fi

- name: Get deployment URLs
  id: get-urls
  run: |
    ADMIN_URL=$(aws cloudformation describe-stacks \
      --stack-name MyStackApp \
      --query 'Stacks[0].Outputs[?OutputKey==`AdminURL`].OutputValue' \
      --output text)
    USERS_URL=$(aws cloudformation describe-stacks \
      --stack-name MyStackApp \
      --query 'Stacks[0].Outputs[?OutputKey==`UsersURL`].OutputValue' \
      --output text)
    COMMUNITIES_URL=$(aws cloudformation describe-stacks \
      --stack-name MyStackApp \
      --query 'Stacks[0].Outputs[?OutputKey==`CommunitiesURL`].OutputValue' \
      --output text)
    echo "admin-url=${ADMIN_URL}" >> $GITHUB_OUTPUT
    echo "users-url=${USERS_URL}" >> $GITHUB_OUTPUT
    echo "communities-url=${COMMUNITIES_URL}" >> $GITHUB_OUTPUT

- name: Deployment Status
  if: ${{ success() }}
  run: |
```

```
    echo "Deployment successful!"
    echo "Admin App URL: ${{ steps.get-urls.outputs.admin-url }}"
    echo "Users App URL: ${{ steps.get-urls.outputs.users-url }}"
    echo "Communities App URL: ${{ steps.get-urls.outputs.communities-url }}"

- name: Deployment Failure Notification
  if: ${{ failure() }}
  run: |
    echo "Deployment failed!"
```

Código 20: Fase de validación de *AWS*

Con esto se habrá concluido y, finalmente, se mostrará en nuestro dominio `admin.solidarianid.com`.

5. Conclusiones

Los *pipelines* de *CI/CD* de *SolidarianID* representan una inversión estratégica en la automatización y confiabilidad del proceso de desarrollo. El *CI* garantiza que el código mantenido en *develop* cumple con altos estándares de calidad, mientras que el *CD* automatiza completamente el camino desde el código verificado hasta la ejecución en producción.
Esta arquitectura permite al equipo de desarrollo iterar rápidamente, detectar problemas tempranamente y desplegar cambios con confianza, reduciendo significativamente el tiempo entre desarrollo y disponibilidad para el usuario final. Además, la trazabilidad proporcionada por los *tags* únicos de las imágenes *Docker* y los reportes de *SonarQube* facilita la auditoría y el *debugging* en caso de incidentes.
En conclusión, estos *pipelines* no solo automatizan tareas operativas, sino que establecen de nuestro proyecto *SolidarianID*.

Referencias

[1] Equipo 3. *Repositorio de GitHub del equipo 3*. 2026. URL: https://github.com/fsoladur/SolidarianID.

[2] Amazon Web Services. *Documentación de Amazon Secrets Manager cli*. Accedido: 27-12-2025. URL: https://docs.aws.amazon.com/es_es/cli/v1/userguide/cli_secrets-manager_code_examples.html.

[3] Amazon Web Services. *Documentación de S3 cli*. Accedido: 27-12-2025. URL: https://docs.aws.amazon.com/cli/latest/reference/s3/.

[4] Amazon Web Services. *Documentación de System Manager cli*. Accedido: 27-12-2025. URL: https://docs.aws.amazon.com/cli/latest/reference/ssm/.